



This article is part of the topic “Action Games as Cognitive Science Paradigm,” Wayne D. Gray (Topic Editor). For a full listing of topic papers, see: [http://onlinelibrary.wiley.com/journal/10.1111/\(ISSN\)1756-8765/earlyview](http://onlinelibrary.wiley.com/journal/10.1111/(ISSN)1756-8765/earlyview).

Mario Becomes Cognitive

Fabian Schrodtt, Jan Kneissler, Stephan Ehrenfeld, Martin V. Butz

Department of Computer Science and Department of Psychology, Eberhard Karls University of Tübingen

Received 27 June 2016; received in revised form 15 November 2016; accepted 15 November 2016

Abstract

In line with Allen Newell’s challenge to develop complete cognitive architectures, and motivated by a recent proposal for a unifying subsymbolic computational theory of cognition, we introduce the cognitive control architecture SEMLINCS. SEMLINCS models the development of an embodied cognitive agent that learns discrete production rule-like structures from its own, autonomously gathered, continuous sensorimotor experiences. Moreover, the agent uses the developing knowledge to plan and control environmental interactions in a versatile, goal-directed, and self-motivated manner. Thus, in contrast to several well-known symbolic cognitive architectures, SEMLINCS is not provided with production rules and the involved symbols, but it learns them. In this paper, the actual implementation of SEMLINCS causes learning and self-motivated, autonomous behavioral control of the game figure Mario in a clone of the computer game Super Mario Bros. Our evaluations highlight the successful development of behavioral versatility as well as the learning of suitable production rules and the involved symbols from sensorimotor experiences. Moreover, knowledge- and motivation-dependent individualizations of the agents’ behavioral tendencies are shown. Finally, interaction sequences can be planned on the sensorimotor-grounded production rule level. Current limitations directly point toward the need for several further enhancements, which may be integrated into SEMLINCS in the near future. Overall, SEMLINCS may be viewed as an architecture that allows the functional and computational modeling of embodied cognitive development, whereby the current main focus lies on the development of production rules from sensorimotor experiences.

Keywords: Cognitive architecture; Cognitive science and games; Artificial intelligence; Self-motivated behavior; Learning; Event schemata; Symbol grounding; Language grounding

1. Introduction

Despite the ground-breaking paper of Terry Winograd over 40 years ago (Winograd, 1972), in which he claimed that a computer program understood the English language—at least while talking about a blocks world, AI programs that develop or exhibit a real understanding of a natural language remain elusive. In the attempt to solve the Turing test, chat bots cheat by producing suggestive comments or by side-stepping meaningful questions in order to fool the users into believing that they are talking to an actual person (Levesque, 2014; Turing, 1950). Similarly, most game agents remain on rudimentary, hard-coded intelligence levels, the systematics of which are typically quickly unraveled.

Concurrently with Winograd, Allen Newell and Herbert Simon proposed that *production rules* may enable the modeling of human problem solving (Newell & Simon, 1972)—leading to the development of the cognitive architectures Soar, ACT-R, and ICARUS (Anderson, 1993; Anderson et al., 2004; Langley & Choi, 2006; Langley, Laird, & Rogers, 2009; Newell, 1990; Rosenbloom, Laird, & Newell, 1992). In 1973, Newell wrote a commentary for a symposium in which he emphasized that three courses of actions may lift cognitive science from the study of particular behavioral phenomena with mostly paradigm-driven experimentations to encompassing theories of cognition. First, he proposed to develop complete processing models, akin to production systems, with the aim of modeling “the detailed control structure” present in our brain (Newell, 1973, p. 301f). Second, he proposed to “accept a single complex task and do all of it,” that is, “to focus a series of experimental and theoretical studies around a single complex task” (Newell, 1973, p. 303), such as chess. Third, he proposed “to stay with the diverse collection of small experimental tasks [...] but to construct a single system to perform them all” (Newell, 1973, p. 305).

In this paper, we introduce an embodied cognitive architecture, whose components and overall functionality are motivated from insights in both artificial intelligence and cognitive science. We call the architecture SEMLINCS, which is a loose acronym for *SEMan-tic, SEnsory-Motor, SElf-Motivated, Learning, INtelligent Cognitive System*. The current main goal of SEMLINCS is to investigate by computational means how a control structure—such as the human brain—can learn suitable production rule-like conceptual structures from continuous sensorimotor experiences in a self-motivated manner. Moreover, SEMLINCS investigates how the developing, conceptual structures can be used to interact with the world in a self-motivated, goal- and information-driven manner. To evaluate the abilities of SEMLINCS, a first implementation controls the behavior of the game agent Mario in its Super Mario Bros. virtual game environment (Fig. 1), within which Mario learns conceptual knowledge from the gathered sensorimotor information.

SEMLINCS thus tackles all three propositions of Newell to a certain extent. It offers a somewhat complete processing model, it focuses on a single, but complex task domain, and it performs various tasks within the domain. Moreover, SEMLINCS’s control structure is closely related to Winograd’s original SHRDLU program. However, SEMLINCS learns the rule structures.



Fig. 1. Mario in its Super Mario world.

SEMLINCS is additionally motivated by a recent proposal of a unified subsymbolic computational theory of cognition (Butz, 2016). Information processing and learning is based on formalizations of free energy-based inference (Friston, 2009, 2010). Moreover, fully autonomous, self-motivated, goal-oriented behavior is generated by anticipatory, active inference (Friston et al., 2013). Finally, the theory specifies how predictive, conceptual, compositional knowledge representations (akin to production rules) may be learned by focusing generalization and segmentation mechanisms on the detection of events and event boundaries.

Following the original characterization of an event as “a segment of time at a given location that is conceived by an observer to have a beginning and an end” (Zacks & Tversky, 2001, p. 3), we refer to an *event* as a certain type of interaction that ends with the completion of the interaction, that is, with an *event boundary*. In the Super Mario world, such interactions typically consist of approaching an object from a certain direction and end with establishing contact with the object. As a result, the event boundary marks the end of an event by encoding the encountered changes in both the object and the bodily properties of the Mario agent, such as the destruction of a “simple block” by Mario upon contact from below.

To summarize, the overall aim of SEMLINCS is to explore how conceptual, rule-like structures can be learned from continuous sensorimotor experiences and to what extent these structures are well suited for the generation of human-like behavior and cognition. Thus, in contrast to available production rule systems, SEMLINCS does not require a theorist, who encodes the possible actions that are performed and the responses that are possible (Polson & Lewis, 1990), but it can learn these symbolic encodings from sensorimotor experiences. To be able to develop a fully functional system in the Super Mario Bros. game environment, we provide several helper functions, prior knowledge, and a language interface in order to “bootstrap” the learning

process and to make the cognitive abilities of SEMLINCS accessible to a human user. As a result, our implementation allows for full-blown simulations of conceptual learning, hierarchical goal-directed planning and control, as well as for linguistic user interactions, as illustrated and evaluated below. Interestingly, current limitations of the specified SEMLINCS implementation—discussed at the end of the paper—emphasize the need for the integration of additional cognitive modules and predictive encodings.

When comparing SEMLINCS with recent deep learning approaches, one notices that learning starts from continuous sensors or sensorimotor streams of information in both cases. However, in harsh contrast to deep learning approaches (Mnih et al., 2015; Silver et al., 2016), SEMLINCS develops an explicit production rule-like knowledge structure. This knowledge structure enables the system to speak about and thus to exhibit an “understanding” of the game it plays. Moreover, it enables the generation of strategic, hierarchical, goal-oriented, and self-motivated interactions with the game. The developing “understanding” and the strategic behavioral capabilities come from the fact that SEMLINCS is predictive and goal-oriented in nature, rather than being blindly—largely reactively—driven toward the optimization of particular reward signals (Mnih et al., 2015; Silver et al., 2016).

In the following section, we first provide a general overview of SEMLINCS, motivating its core design choices from a cognitive science perspective. We then detail the actual implementation, again highlighting the core assumptions and briefly sketching out the additional concrete, task-specific mechanisms. The evaluations exhibit unique aspects of SEMLINCS, including its self-motivated, goal-directed behavioral abilities and the development of conceptual, discrete production rules from continuous sensorimotor experiences or from linguistic information. The final discussion reflects on the insights gained from our modeling efforts, highlights important current system limitations and resulting possible further enhancements, and finally relates the achievements back to Newell’s original proposition.

2. Cognitive architecture SEMLINCS

SEMLINCS combines insights from artificial intelligence and machine learning with psychology, biology, linguistics, computational neuroscience, and cognitive modeling. It is a predictive architecture, which continuously monitors its unfolding sensorimotor experiences and learns to segment these experiences into meaningful, event-oriented interaction schema rules. Thus, SEMLINCS faces the symbolic and semantic grounding challenges, that is, how higher level cognition may be grounded in sensorimotor experiences (Harnad, 1990), taking an embodied, developmental approach. We now first motivate the overall architecture and its key components and then provide further details about the realization of the individual components and their actual implementation in the Super Mario world.

2.1. Motivation and overview

Over the last two decades, cognitive science has emphasized that cognition is *grounded*, or *embodied*, in sensory-, motor-, and body-mediated experiences, which humans, and other adaptive animals, gather in their environment (Barsalou, 2008). This essentially means that our conceptual understanding of and thoughts about our environment develop ontogenetically, leading to progressively more complex but rather systematized, interactive cognitive structures. Clark (2013) has hypothesized that this grounding of cognition can be accomplished by applying principles of predictive coding and free energy-based inference.

Indeed, over the last two decades, it has also been proposed numerous times that brains are generative, predictive coding systems (cf. e.g., Bar, 2009; Friston, 2003, 2009; Rao & Ballard, 1999). In such systems, learning predominantly focuses on the differences between predicted and actual sensory information, including respective precision estimates. Moreover, behavior is inherently goal-directed (Butz, 2008; Butz, Sigaud, & Gérard, 2003; Friston, 2003, 2009; Hoffmann, 1993; Prinz, 1990). The free energy-based inference principle subsumes predictive coding methods and can account for neural activity adaptations, neural learning, as well as active epistemic and motivation-driven inference, offering a potentially unified brain principle (Friston, 2009, 2010).

Concurrently with these developments, it has been suggested that our mind processes *event boundary signals* in a unique manner. The event segmentation theory (Zacks & Tversky, 2001; Zacks, Speer, Swallow, Braver, & Reynolds, 2007) suggests that humans learn to segment the continuous sensorimotor stream into *event codes*, which are also closely related to the common coding framework and the theory of event coding (Hommel, Müsseler, Aschersleben, & Prinz, 2001; Prinz, 1990). Zacks and Tversky (2001) proposed that such event codes should be very well-suited to being integrated into event schema-based rules, which are closely related to production rules (Newell & Simon, 1972) and rules generated by anticipatory behavior control mechanisms (Butz & Hoffmann, 2002).

At this point, however, the available theories and computational models are spelled out only on a rather general level. Actual implementations are either missing or are hardly scalable and are restricted to limited domains (Bastos et al., 2012; Friston, Mattout, & Kilner, 2011; Friston et al., 2015; Zacks, Kumar, Abrams, & Mehta, 2009). Most important, however, it remains largely unresolved how event codes and symbolic abstractions can be learned from continuous sensorimotor experiences. A recent, unifying theory (Butz, 2016) offers an option to foster such abstractions. Among other things, the theory emphasizes that event and event boundary detection mechanisms, which can be derived from the free energy principle by means of processing significantly surprising events in a distinct manner, may foster the learning of event schemata by means of which hierarchical planning and conceptual reasoning become possible (Gumbsch, Kneissler, & Butz, 2016).

SEMLINCS integrates these theoretical lines of thought and proposes a novel, unifying, predictive cognitive architecture, which learns symbolic conceptual structures in a self-motivated manner from continuous sensorimotor experiences. As a result, it grounds

the developing production rules and involved symbols semantically in the gathered sensorimotor experiences. Thus, SEMLINCS is generally complementary to production rule systems, offering a fundamental way to create symbols and production rules from (continuous) sensorimotor experiences.

From a psychological perspective, it can be said that SEMLINCS acts based on the ideomotor principle (Herbart, 1825; James, 1890; Stock & Stock, 2004) and learns based on the reafference principle (Holst & von Mittelstaedt, 1950). To bootstrap learning, we equip the actual implementation with low-level, sensorimotor planning abilities (A* search) as well as with temporal predictive, sensorimotor forward models about the low-level consequences of motor actions, which are simulated key presses in the Super Mario world, on its body and its environment. Note, however, that in the more general case SEMLINCS would begin with learning sensorimotor contingencies by means of initial motor and goal-babbling, observing the consequences of its behavior and optimizing the resulting developing predictive knowledge structures. Various methods from computational cognitive science research are available that are able to learn such sensorimotor forward models (cf. e.g., Butz, Herbort, & Hoffmann, 2007; Ehrenfeld, Herbort, & Butz, 2013; Haruno, Wolpert, & Kawato, 2001; Jordan & Rumelhart, 1992; Kneissler, Drugowitsch, Friston, & Butz, 2015; Rolf, Steil, & Gienger, 2011; Wolpert & Flanagan, 2003; Wolpert & Kawato, 1998.)

By means of the (in this case provided) low-level forward model, SEMLINCS continuously predicts sensory consequences. It begins to develop associations between actions and their consequences once unexpected interaction event boundaries are detected, such as the disappearance of a box or the increase in its coin count. Combined with the need to maintain internal homeostasis, the information processing mechanisms of the architecture yield self-motivated, goal-directed behavior (Butz, Shirinov, & Reif, 2010). From a computational neuroscience perspective, the unfolding computations are closely related to active free energy-based inference mechanisms (Butz, 2016; Friston et al., 2013, 2015).

An overview of the implemented architecture and the predictive flow of information is provided in Fig. 2. SEMLINCS essentially integrates principles of self-motivated behavior, event-oriented learning and knowledge structuring, and goal-oriented hierarchical planning and control. Furthermore, the addition of a speech system illustrates that the developing knowledge structures provide semantic knowledge, which can be communicated linguistically in a straightforward manner.

2.2. Motivational system

SEMLINCS equips an agent with a motivational system, which consists of a set of dynamic reservoirs. The further away a current reservoir state is from its optimal homeostatic state, the more likely is the invocation of a drive to achieve those intrinsic or extrinsic goal event boundaries that—as the system believes—are expected to change the drive-invoking reservoir state toward homeostasis. As a result, the agent's behavior becomes “intentional,” focusing on the generation of self-rewarding event boundaries. By means of the developing schema rule knowledge, SEMLINCS agents become able to plan

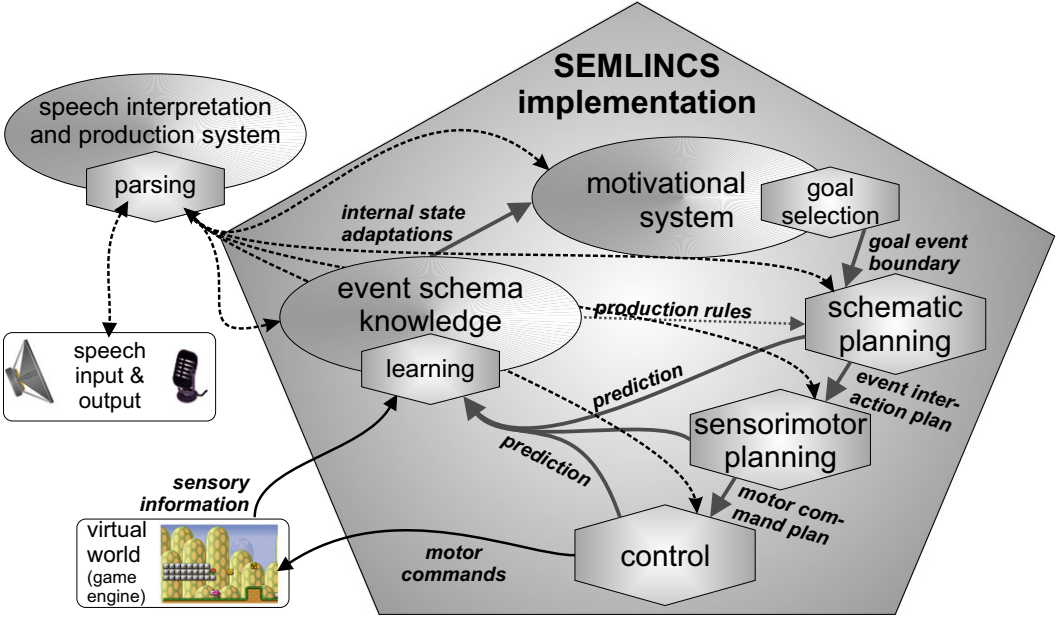


Fig. 2. Sketch of the SEMLINCS implementation, including the interactions with the user by means of the speech interface and with the virtual environment via simulated motor commands and sensory feedback. Once the motivational system has selected a goal event boundary, it is translated into a series of environmental interactions. The motor commands are subsequently planned, attempting to invoke the desired goal event boundary and—in the case of success—lowering the respective motivational drive. In the process, the agent progressively learns more about events, updating its event schema knowledge. As a result, increasingly clever environmental interactions are accomplished.

complex, goal-directed sequences of environmental interactions. From a broader perspective, the system essentially combines active inference mechanisms with principles of autopoiesis, creating a self-sustaining system (Butz, 2016; Friston et al., 2013; Maturana & Varela, 1980).

Our concrete implementation of the motivational system is based on Butz et al. (2010). Reservoir states away from a predefined “homeostatic” state cause an “urge” to select those goal event boundaries that are believed to push the reservoir state back toward homeostasis.

Formally, the current $urgen_d^t$ of a particular drive (or motivation), indexed d , can be defined as a sigmoid function of the current, respective *reservoir states* s_d^t :

$$u_d^t = \frac{1}{1 + \exp(\kappa_d \times (s_d^t - m_d))} \Big|_{s_d^{\min}}^{s_d^{\max}}, \quad (1)$$

where κ_d determines the steepness of the sigmoid function and m_d the inflection point. The drive thus increases nonlinearly from zero to one with increasingly lower reservoir

state values. The urge is rectified at minimum and maximum values $s_d^{\min}$ and $s_d^{\max}$, respectively. Corresponding sigmoids are shown in Fig. 3. For example, the reservoir state of curiosity s_1^t determines the corresponding urge nonlinearly: While a reservoir value above 10 leads to urge values below 0.5, the urge nonlinearly increases with progressively lower values, converging toward the maximal urge 1 when reaching the empty reservoir.

Reservoir states s_d^t themselves behave dynamically governed by the following, general equation:

$$s_d^t = f_d(s_d^{t-1}) + r_d^t, \quad (2)$$

where $f_d(s_d^{t-1})$ implements the dynamics of the reservoir, for example, a linear or random decrease of the state over time. The reward r_d^t represents extrinsic or intrinsic changes in the reservoir state, triggered by the observation of specific, motivation-relevant, parameterizable game event boundaries, such as collecting or creating an item, increasing the discrete health state by one unit, or destroying an obstacle. For the specific event boundaries considered by our implementation, see Table 1.

We used four basic motivations: “curiosity,” “wealth,” “progress,” and “health.” *Curiosity* reflects the agent’s incentive to learn about events, event boundaries, and conceptualizing event-encoding production rules (i.e., event schemata). *Progress* is Mario’s predisposition to head to the right side of the level (in order to complete it). The covered horizontal distance manipulates the drive’s dynamics. The *wealth* reservoir represents the drive to gather coins. Finally, *health* reflects the need for good health and survival, where the “health” of Mario is simply reflected by a discrete value, which can be increased by consuming “mushrooms” or “fire flowers,” and which decreases when bumping into an enemy from the sides. An exemplar configuration of the respective urge functions and a momentary state of the agent’s motivational system is shown in Fig. 3. Appendix A details the concrete implementation of the reservoir dynamics further.

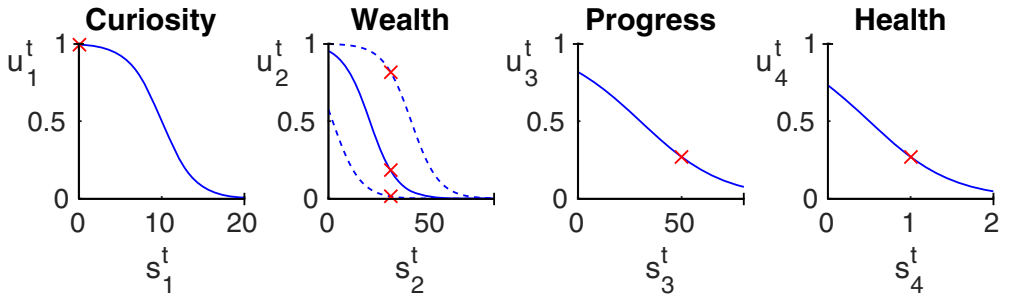


Fig. 3. Exemplar configuration of motivational system. Functions in blue show the nonlinear dependency of urges on the reservoir states, while red crosses mark the current state and urge. Currently, the urge to explore new items (curiosity) is the strongest. For wealth, three different parameterizations of the reservoir’s inflection point are shown, illustrating that the same reservoir state can result in different urge values.

Table 1

Game event boundaries that trigger changes in the reservoir states. Reservoirs tagged with (r) feature random reservoir dynamics, while (c) indicates the absence of internal dynamics. The rewards added to the reservoir states (r_d^t) are either positive or negative, while the reward for new knowledge follows from the absolute update of the knowledge entries (see section 3). Progress is rewarded (or punished) according to the distance an agent has covered toward the end of the level

Reservoir	Game Event Boundary	Reward r_d^t
1) Curiosity (r)	OBJECT_INTERACTION	$\max(\sum \rho \Delta P^t , 0.1)$
2) Progress (r)	POSITION_REACHED	Δx
3) Wealth (c)	COIN_INCREASE	5
	NEW_GOAL_SELECTED	-1
4) Health (c)	HEALTH_INCREASE	1
	HEALTH_DECREASE	-1

Based on the current state of the motivational system, the agent makes a probabilistic choice on which goal event boundary to pursuit. Formally, first, an *executive reservoir* with index $e \in D^t$ is selected with a probability proportional to the current motivational urges:

$$P(e = d) = \frac{u_d^t}{\sum_{d'} u_{d'}^t}, \quad (3)$$

where $d' \in D^t$ iterates over the set of urges D^t that—as the system believes—may currently be altered by a particular entity interaction. An event boundary that is believed to fill the executive reservoir is then activated as the next goal.

Before being able to plan sequences of tangible motor commands to invoke this event boundary, the agent first plans a conceptual level by translating the goal event boundary into one or more environmental interaction events. To accomplish this event translation process, schematic knowledge about interactions and their preconditions is required.

2.3. Schematic knowledge

Schematic knowledge enables the agent to plan and reason about its environment on an abstract, conceptual level without planning actual motor commands. SEMLINCS learns this knowledge. It continuously predicts bodily and environmental state changes by means of the currently available sensorimotor and event-based temporal forward models. When an event boundary is detected, event schema learning takes place (Butz, 2016). Besides its close relation to free energy-based inference, the underlying principle is also closely related to the reafference principle (Holst & von Mittelstaedt, 1950)—essentially processing registered exafferences in a principled manner. As a result, production rule-like, sensorimotor-grounded precondition interaction event boundary schema rules develop.

2.3.1. Schema structure

In our actual implementation of SEMLINCS, event schemata develop, which probabilistically predict an event boundary given the specified preconditions are satisfied and a particular object interaction is executed. Fig. 4 shows two examples. The *preconditions*

can contain variables about the agent's current bodily state and the state of the interaction target. For example, in the game, Mario's "health level"—an integer number—determines if it is currently "small" or "large." Because Mario can only destroy blocks when it is large, typically a precondition develops when predicting the effects of an object destruction by a collision with a block from below. Fig. 4 shows a resulting rule, which probabilistically predicts the possible outcomes of an interaction with a "question mark block." In our implementation, the *interaction* part of the rule focuses on entity interactions (including objects, enemies, and other agents), because these are the only interactions during which unexpected event boundaries can occur. Seeing that the result of an interaction depends on the direction from which Mario collides with the object, we clustered all types of interactions into four collision direction clusters, that is, colliding with the object from the upper, lower, left, and right side.

As stated above, generally speaking, *event boundaries* comprise all those perceivable changes in the environment that the agent does not predict by means of its sensorimotor knowledge. The implemented detection mechanism for event boundaries is generally generic, because it is based on the principle of identifying significant, lasting changes (Butz, 2016; Butz, Swarup, & Goldberg, 2004; Gumbsch et al., 2016). As a result, event boundaries comprise (a) intrinsic changes in the agent's condition (health, wealth), (b) extrinsic changes in the environment (appearance or disappearance of specific objects, state changes in specific non-player characters), and (c) particular interactions between the agent and its environment (colliding with an object, reaching a goal position). Note that the "progress" event boundary is hard-coded, invoking the boundary when reaching the right side of the screen. In the future, it may also be learned by registering the resulting level advancement (i.e., registering level scrolling events).

2.3.2. Schema learning

In our concrete implementation, the agent is initially able to anticipate repulsions from solid objects, but it is unable to predict any other event boundary caused by the

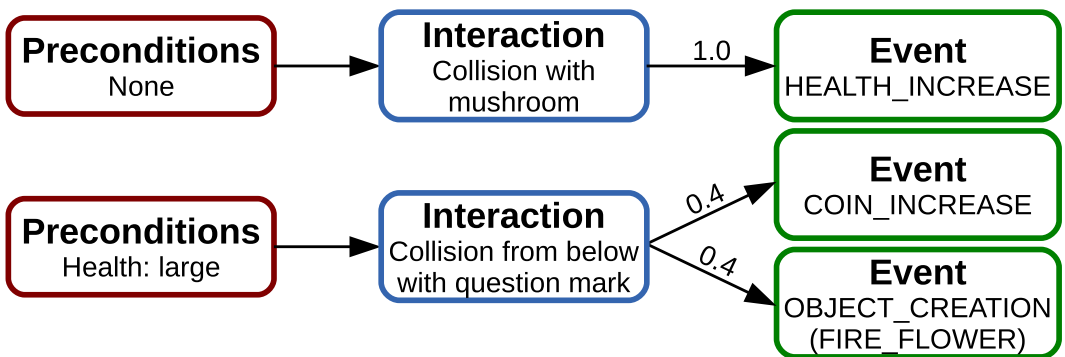


Fig. 4. Schematic knowledge base example. Here, the agent has learned that colliding with a mushroom will result in a health increase. Moreover, there is uncertainty about the outcome of colliding with a question mark block from below when the current health status is "large."

interaction with an object. The agent continuously compares its sensorimotor predictions with its perceptions while interacting with the environment. When a significant difference is perceived between predictions and perceptions, the agent processes this difference as an event boundary. If the corresponding event schema is already encoded in the knowledge base, it is updated. Otherwise, a new event schema is created.

Event codes are learned under the assumption that all game event boundaries are on the one hand conditionally independent of each other, but on the other hand dependent on the immediate preconditions and interactions defined above. Hence, the following learning rule can be applied, which follows Bayesian statistics:

$$\Delta P^t(e_a|p_b, i_c) = \eta_{b,c}^t \times p_b^t \times i_c^t \times (e_a^t - P^{t-1}(e_a|p_b, i_c)), \quad (4)$$

where e_a^t, p_b^t and $i_c^t \in \{0, 1\}$ indicate if specific event boundary a , preconditions b , and interactions c are observed at time step t . The learning rate $\eta_{b,c}^t$ is adapted dynamically over time for each rule, dependent on the number of event observations of that type. A lower bound ensures the dynamic adaptation of each rule to novel circumstances (Schrodtt & Butz, 2016). This method ensures that the conditional probabilities are adapted quickly in the beginning. Later on, they approximate a moving average.

Note that the exemplar schema rules shown in Fig. 4 probabilistically predict certain effects given particular preconditions and interactions. For example, when the agent collides with the bottom edge of a question mark block, the effect of the collision is expected to be either an increase in the agent's coin count, indicated by the event boundary COIN_INCREASE, or the creation of a fire flower, indicated by the event boundary OBJECT_CREATION(FIRE_FLOWER). Additionally, the rule implies that the system believes that no change will occur with a probability of 36%.

This knowledge can be learned incrementally, for example, in the following manner. Assume an implicit initialization of all knowledge entries to $P^t(e_a|p_b, i_c) = 0$ and no previous observations. The initial knowledge thus expects that the collision with a question mark block has no effect. When the creation of a fire flower (event boundary e_1) is observed for the first time upon the collision with a question mark block from the bottom edge, the event probabilities are updated by $\Delta P^t(e_1|p_b, i_c) = 1$, because $e_1^t = p_b^t = i_c^t = \eta_{b,c}^t = 1$ for the current condition b and interaction c ($\eta_{b,c}^t = 1$). When, upon a second interaction (then with $\eta_{b,c}^t = 0.5$) with the question mark, the coin count changes instead (event boundaries $e_1 = 0, e_2 = 1$), the knowledge updates will be $\Delta P^t(e_1|p_b, i_c) = -0.5$ (because a fire flower was expected, but not observed) and $\Delta P^t(e_2|p_b, i_c) = 0.5$ (because a coin increase was observed, but not expected), with $\eta_{b,c}^t = 0.5$. After that, both events are believed to be equally likely ($P^t(e_1|p_b, i_c) = P^t(e_2|p_b, i_c) = 0.5$).

In addition to this incremental Bayesian learning, generalization mechanisms are applied. The actual implementation generalizes preconditions over the health level and interactions over all possible directions of collision by updating all respective, currently inapplicable schema rules using a small learning rate. Moreover, when a particular

condition or interaction is unknown, the architecture generalizes them selectively with an average learning rate. This is particularly the case when schematic knowledge is provided linguistically (see Section 2.5 for details).

2.4. Planning and behavior

To invoke rewarding events, hierarchical reasoning, action planning, and control unfold in a goal-directed manner on the probabilistic, event-schematic level and on the sensorimotor level. The planning and reasoning mechanisms of SEMLINCS are based on the principle of active inference (cf. Butz, 2016; Friston et al., 2013). As a result, SEMLINCS considers the execution of any possible object interaction, anticipates the expected results, and evaluates them with respect to its current internal motivational drives. Thus, the motivational drives essentially invoke higher level event boundaries as goals, which start the conceptual planning routines, which in turn invoke a sequence of interaction events as successive goals. Lower level planning then determines the best sequence of continuous sensorimotor interactions for accomplishing the targeted interaction events. SEMLINCS thus implements a higher level schematic planning and reasoning mechanism, which interacts with lower level sensorimotor planning and resulting actual motor control (cf. Fig. 2).

In the following sections, we explain how the two levels of planning are realized in our current implementation. Note that while the two levels can be separated rather easily in the Mario world, SEMLINCS is generally neither restricted to two levels nor to clear-cut separations between the levels.

2.4.1. Schematic planning

Given a goal event boundary, which is decided upon based on the agent's motivations as detailed above, the agent attempts to invoke the particular goal by means of executing a suitable sequence of motor commands. In the Mario world, motor commands correspond to the execution of simulated key presses—exactly how an actual player would control Mario. When schematic knowledge exists about the goal event boundary, schematic planning may be invoked to resolve possible preconditions. This schematic planning is currently implemented by a stochastic Dijkstra algorithm, which plans a path toward an event boundary given its current schematic knowledge. Starting at the currently observed condition, it simulates known interactions with reachable objects and samples the possible effects stochastically, dependent on the probabilities specified in the involved schematic rules. As a result, event chains can be simulated. The cost function is based on the likelihood of the learned event schemata, such that schemata with a high probability of success are considered first.

The reachability of interactions is estimated using a potential field (cf. Hwang & Ahuja, 1992), which expands from the agent in all directions along the free space (see Fig. 5). Thus, it excludes objects in the environment that cannot be reached (e.g., because they are blocked by other objects). The reachability estimate is updated during the search when object destructions and health-level-dependent changes in the agent's abilities are

simulated. As a result, schematic planning can anticipate increases in the agent's action options and, thus, interactions with objects that are blocked by destructible blocks can be planned successfully.

The search terminates when the desired goal event boundary is believed to be reached with a sufficiently high likelihood. In the current implementation, the likelihood directly depends on the urge of the currently active drive:

$$p(e_g^t | (p_1, i_1), \dots, (p_n, i_n)) \geq 1 - u_d^t, \quad (5)$$

where $p(e_g | i_1, \dots, i_n)$ is the estimated probability for invoking the goal event boundary e_g by means of a series of interactions $i_1, \dots, i_n$, and u_d^t is the urge of the current executive reservoir d . Thus, the algorithm translates a desired event boundary into a sequence of environmental interactions, which are believed to most likely fill the executive reservoir. By linking the urge with the likelihoods, low urges will invoke confident interactions, while high urges can also trigger uncertain or unconventional interactions. If planning is not successful because the current knowledge is insufficient or because the success of the best found interaction plan is believed to be highly unlikely, the current event boundary is considered unreachable and another goal event is chosen by the motivational system.

For example, assume the knowledge provided in Fig. 4, where question mark blocks release coins with a certain probability only if the agent's health status is "large." If the selected goal is to increase the agent's number of coins by invoking the event COIN_INCREASE, this precondition has to be fulfilled. If however, the agent is currently "small," the difference between the precondition and the actual condition is translated into another



Fig. 5. A potential field heuristic is used to determine regions and objects that are currently reachable. In the figure, the potential field expands from the goal position (star at the bottom right) to the agent (top center) along the free space. The shortest distance of each expanded path to each reachable position is color-coded from blue (short distance) to red (long distance). Unreachable areas are excluded from the search space (gray).

intermediate goal event boundary, which is the event boundary HEALTH_INCREASE. Considering the exemplar rules in Fig. 4, the agent currently believes that the latter event boundary can be achieved by colliding with a mushroom. As a result, given sufficiently accurate schematic knowledge, the agent may decide to first collide with a mushroom and then go for the question mark block.

Note that schematic planning is invoked when a particular event boundary is desired. In the case of the “progress” motivation, the desired POSITION_REACHED event is translated into reaching the position above a solid ground object at the right side of the currently visible scene. On the other hand, when the executive reservoir is the “curiosity” motivation, the goal, that is the desired object interaction event boundary, is determined by the currently available schematic knowledge. The implementation chooses the object interaction about which the knowledge about its possible effects is most uncertain. Note also that schematic planning is able to realize the need to destroy objects that block a pathway—thus expanding the reachable space—due to the forward simulation during conceptual planning and the employed reachability heuristic. Similarly, schematic planning may anticipate a necessary change in the health state to be able to destroy certain blocks,¹ such that the agent may strive toward the interaction with a mushroom, followed by the destruction of a block, followed by eventually moving to the desired position on the right side of the screen (thus making the desired “progress”).

2.4.2. Sensorimotor planning and control

After schematic planning, a plan is available in the form of a sequence of interactions. This sequence then needs to be translated into lower level motor programs, which ultimately generate motor activities. In our current implementation, the planning of a tangible sequence of actual motor commands is realized by a local, deterministic forward simulation with respect to keystrokes, in combination with an augmented A* search. The temporal forward model only anticipates repulsions from obstacles and positional changes of the agent. We assume here that the agent has already learned this model. Complementary to this model, the learned schematic forward model anticipates the effects of the pre-planned interactions to simulate changes in the environment. The cost function of the A* algorithm specifies distinct costs for each possible key to avoid unnecessarily complex action plans. The involved heuristic estimate to the goal uses the same potential field that is used to determine reachability, only that it expands inversely from the selected goal object toward the agent’s position (see Fig. 5). If successful, sensorimotor planning provides a precise motor plan in the form of a sequence of keystrokes that leads to the desired goal interaction, given that the environment behaves deterministically. Once the necessary sequence of keystrokes for an interaction is determined, it is executed while the next interaction is planned in parallel.

2.5. Speech comprehension and generation

As acknowledged from various perspectives, event-based interaction structures are very well suited to be coupled with a linguistic, grammatical system (Dominey, 2013; Knott, 2012; Pastra & Aloimonos, 2012; Wörgötter, Agostini, Krüger, Shylo, & Porr, 2009).

SEMLINCS is designed such that the developing event-oriented schematic knowledge structures and the motivational system can be coupled with a natural language processing component. By these means, it is possible to probe the knowledge of the system as well as to instruct the system verbally. The main purpose of the speech system is thus two-fold: first, to illustrate to what extent the system's knowledge is compatible with syntactic structures; second, to enable meaningful verbal interactions with the system.

Our current implementation links the internal states and the acquired semantic knowledge of SEMLINCS with linguistic syntax—adhering to the principles of a parallel language architecture (Jackendoff, 2002). The syntax comes in the form of a pre-structured, tagged, context-free grammar. The semantic knowledge develops in the form of the learned event schemata, as detailed above. The resulting linkage between syntax and semantics enables the generation of meaningful conversations and instructions about the agent's current knowledge of the game world and its internal motivational states. In particular, schematic knowledge can be provided verbally. Moreover, orders can be given, such as to execute simple movements, to achieve a particular goal (such as “go to a coin”), or to be motivated in a particular manner.

Speech recognition is realized via CMU Sphinx-4 by Carnegie Mellon University. Sphinx-4 is based on hidden Markov models and supports the specification of dictionaries and context-free grammars in the Java Speech Grammar Format (JSGF) (Lamere et al., 2003). JSGF allows the specification of a set of tags for each terminal or non-terminal symbol in a grammar. In our implementation, each tag A(B) consists of a *keyword* A and a *parameter* B. The tag structure was developed by hand, creating a map to the developing schematic knowledge structure. Depending on the keyword, a tag can represent a specific precondition, an interaction, or an event boundary. This enables the provision of new as well as the querying of current event-related knowledge. Similarly, the speech tags create a map to the agent's motivational states, to various goal interactions, as well as to simple movement commands, which enables the manipulation of its motivations, its current goals, as well as its simple movements. During speech recognition, each recognized sentence corresponds to a definite path through the graph spanned by the defined context-free input grammar, as exemplified in Fig. 6.

While semantic speech comprehension is thus realized by defining a mapping from sentences to tags, using JSGF grammars, speech generation is realized by inverting this mapping. The underlying tag structure of the output grammar is very similar to the input grammar. The inverse mapping essentially allows the verbalization of learned schematic knowledge by the agent. Moreover, information about the agent's internal state can be verbalized. Because a single, symbolic meaning can usually be expressed in several ways, the output mapping provides numerous sentences for a single tag set. When a specific output is generated, one of the sentences is chosen randomly to account for linguistic diversity.

We do not provide further details about the implemented language system, because it is not the most critical part of SEMLINCS. Rather, it serves the purpose of highlighting that the developing knowledge structure can be easily verbalized, and that knowledge can

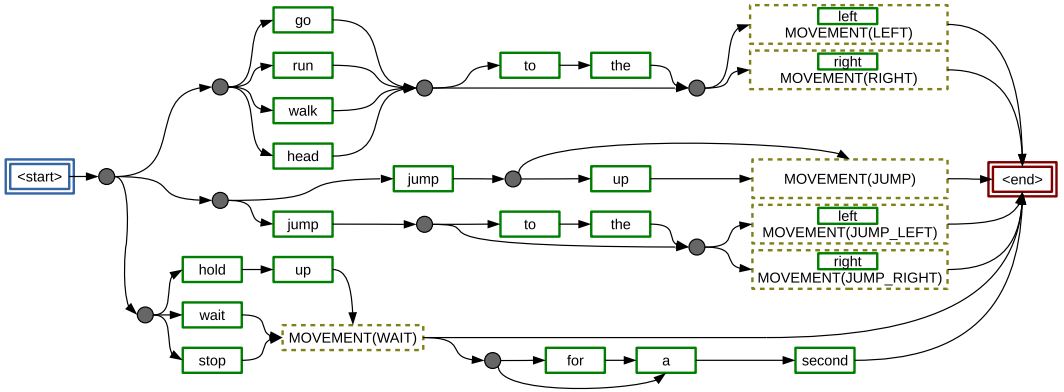


Fig. 6. Tagged subgraph reflecting a simple movement command grammar. Blue and red indicate starting and endpoint, respectively. Option branches are shown by gray circles, while terminal symbols are displayed in green. All tagged entities are shown as yellow dashed boxes. The semantics of a recognized sentence can be determined by collecting the respective tags.

be either provided verbally or learned from sensorimotor experiences. Moreover, it makes the implemented system much more attractive for human users.

3. Evaluations

We now evaluate several behavioral abilities of the learning agent, which is controlled by the SEMLINCS architecture. In particular, we first provide several examples of verbal user interactions with the agent, starting without any event schema knowledge. We illustrate the diverse options for gathering knowledge, the conversational abilities, and the knowledge-dependent, that is, belief-dependent planning mechanism. Moreover, we evaluate and analyze the autonomous performance of the agent, dependent on particular motivational reservoir state settings. For an introduction to the system and current advancements with respect to its social abilities (the latter we do not detail in this paper), please refer also to our online video material.^{2,3} An executable program is available upon request for teaching and research purposes.

3.1. Options for knowledge acquisition and resulting behavior

Let us first look into the different options with which the agent can acquire knowledge about its environment, and how it can then use this knowledge to reach goals that are initially believed to be unreachable. Let us consider the exemplar situation shown in Fig. 7. Mario has gathered no event schema knowledge so far and it is instructed to “make progress”; that is, it is supposed to reach the right edge of the screen while neglecting its other motivations.

Notice that in the shown situation (Fig. 7), the path is blocked by several golden obstacles, which are called “simple block,” and which can only be destroyed when Mario’s

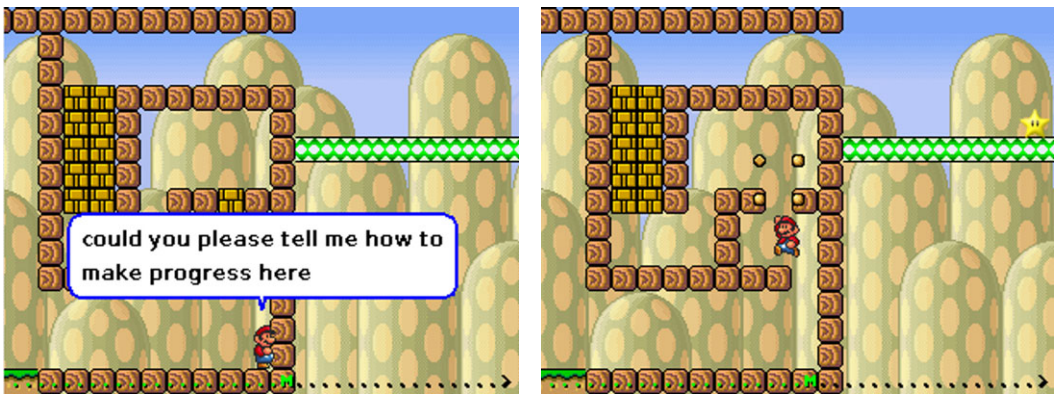


Fig. 7. The agent has the motivation to make progress, but the path is blocked. It is able to learn to anticipate the destruction of the obstacles in several ways.

internal health value is “large,” that is, 2 or higher. Due to the instructed focus on “progress” and the lack of knowledge, Mario will not be able to plan a path through these obstacles. The user may help the Mario agent to solve the problem in several ways:

1. The user can provide it with the knowledge that this type of block is destructible, by stating, for example: “Mario, if you hit a simple block from below, then it is destroyed.”
2. The user can enable Mario’s curiosity drive, by instructing, for example, “Mario, learn more!” This will stimulate Mario’s interest in testing the effect of specific interactions with unknown objects by increasing its curiosity motivation. Because Mario does not know that the simple block can be destroyed when colliding with it from below, Mario is likely to try this interaction.
3. The user can instruct Mario to collide with the simple block from the bottom, for example, by stating: “Mario, hit a simple block from the bottom!” The Mario agent will then process the destruction of the object by means of the schema learning mechanism, storing the detected event in its schema knowledge base.
4. The user can instruct Mario to jump by simply stating “Mario jump” when Mario stands below a simple block—in which case Mario will also notice the event and store it in its event schema knowledge base.

All four means lead to the storage of the event schema rule: “collision from below with simple block” → “object destruction of simple block.” As a result, Mario can then follow the instruction to make progress, because it then knows that the destruction of the path-blocking “simple blocks” will open up a path to reach the platform on the right side of the screen. Mario is thus able to gain the same knowledge by means of the event-oriented processing of own sensorimotor experiences or by means of processing the corresponding linguistic information provided by the user.

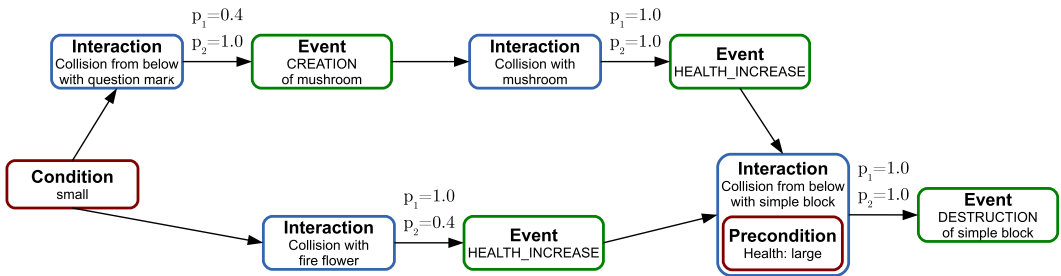


Fig. 8. In the visualized situation, Mario first needs to become large in order to be able to destroy the “simple block.” Mario has two options to achieve this: either he takes the fire flower or he jumps into the question mark block from below and then takes the consequently appearing mushroom on top of the question mark block. The possible two reasoning pathways are shown below the situational screenshot. The probability values indicate two possible belief states, which cause Mario to choose the lower chain in the former and the upper chain in the latter state, given the goal to destroy a “simple block.”

3.2. Knowledge-dependent probabilistic planning

Schematic planning is dependent on the knowledge currently available, including the conditional probability of success, which is associated with each event schema. In the illustrative example shown in Fig. 8, again, the agent starts without any schema knowledge. The following two instructive dialogs result in the two probabilistic knowledge structures shown below the situational screenshot in Fig. 8. Please also refer to our online video material for the actual video of this interaction.⁴

In the first case, Mario is provided with more certain information about the effects of interacting with a “fire flower:”

Hi Mario
-> HEY BRO

If you hit the question mark from the bottom,
then a mushroom is maybe created.

-> IF YOU SAY SO

If you hit a mushroom, then you get healthy.

-> YES, I GOT THAT

When you hit a fire flower, then you get healthy.

-> YES, I GOT THAT

If you collide with a simple block from the bottom
and you are healthy, then it is destroyed.

-> YES, I GOT THAT

Destroy the simple block.

-> OKAY

... and Mario accomplishes the task by first collecting the fire flower becoming large as a result and then destroying the simple block from the bottom.

In the second case, Mario is provided with the same information (for illustrative purposes with slightly different wording) except for the involved certainties:

Hey Mario

-> HELLO

If you hit a question mark from the bottom, then a mushroom is
certainly created.

-> IF YOU SAY SO

If you hit a mushroom, then you certainly get healthy.

-> I UNDERSTAND

If you hit a fire flower, then you maybe get healthy.

-> I UNDERSTAND

When you collide with a simple block from the bottom and you
are healthy, then it is destroyed.

-> YES, I GOT THAT

destroy the simple block

-> AFFIRMATIVE

... and Mario accomplishes the task by first jumping into the question mark block from the bottom, then collecting the resultant mushroom and becoming large as a result, and finally destroying the simple block. Thus, Mario decides to take the behaviorally slightly more demanding option in this case because it is more certain that it will succeed. This example thus illustrates that Mario does not necessarily take the shortest sequence of actions to reach a certain event boundary. Rather, it chooses the action sequence that is most certain, which is not only an inherent effect of the actually implemented planning procedure but also a consequence of applying active inference techniques.

3.3. Motivation-dependent level completion rate

We finally evaluate the agent's ability to autonomously solve a complete level without any help from the user. For that purpose, we designed an evaluation level (see Fig. 9) with the following elements:

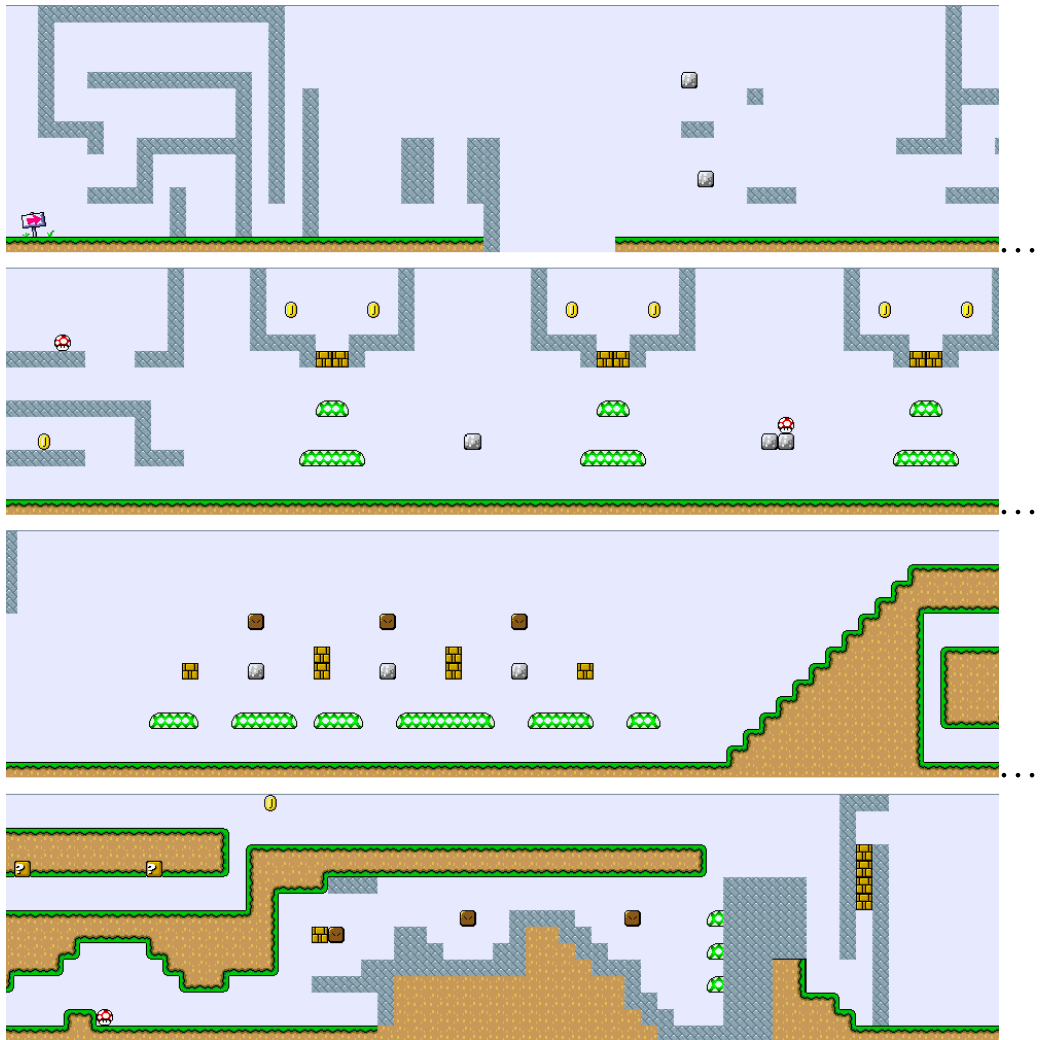


Fig. 9. Level designed for evaluating the effect of different motivation settings on the level completion rate.

- a little maze, which illustrates the sensorimotor path planning capabilities,
- a wide gap, which can only be crossed after climbing a chimney,
- two instances of an initially unknown object (“stone”) without relevance for further progress in the level,
- two relevant objects (a “coin,” and a “mushroom”), with the mushroom being available in the level three times overall,
- three pairs of coins enclosed in boxes that can be accessed only after destruction of “simple blocks,”
- a collection of several objects that Mario can interact with, but without any impact on level completion,

- a branching point where a new object type (“question mark block”) may distract Mario into a dungeon, and finally
- the end of the level, which can be reached via both the dungeon and the upper more direct pathway, and which requires the destruction of the a small wall of simple blocks.

The evaluation has the following objectives. First, the initial maze and wide gap challenge the A*-based sensorimotor planning mechanism. Second, the various objects attract Mario’s curiosity more or less intensively depending on the motivational setting. Third, the dungeon delays Mario from reaching the goal, but it opens up another opportunity to interact with a mushroom. Fourth, the final column of “simple blocks” requires Mario to know how to destroy them, and that the program incorporates this knowledge appropriately into the schematic planning mechanism.

We set Mario’s initial health state to “small” so it can only destroy “simple blocks” after it makes contact with a mushroom—which causes the increase of its health value, being “large” as a result. However, since Mario initially does not know that these objects increase its health, curiosity needs to be strong enough to ensure that such an object interaction is chosen. Note that the evaluation level does not contain any enemies, which could decrease Mario’s health state. As a result, the health drive is irrelevant (in this level) for Mario’s chances of successful level completion.

We set a time limit in terms of the maximum number of simulation steps after which the simulation is terminated and the level is considered not solved when Mario has not reached the end, yet. Furthermore, to illustrate the influence of the motivational settings further, we added an “energy consumption” mechanism: For each planned goal, Mario has to pay one energy unit, whereas for each coin collected in the level, it gains five energy units (coins are thus like food or “energy”). The initial amount of energy units is chosen critically low, such that Mario does not stand a chance of completing the level without collecting coins.

To summarize, these are the ways a simulation run can terminate:

- Successful completion of the level (the right end of the level is reached);
- Starvation (no more goals can be planned);
- Time-out (fixed limit of number of simulation steps reached);
- Falling into an abyss (sensorimotor planning effectively prevents this from happening with very high reliability).

In order to evaluate Mario’s behavior in dependence of its motivational state, we determined its level completion rate under different motivational settings. We fixed the three urges u_{wealth} , $u_{\text{curiosity}}$, u_{progress} throughout the simulation of each run to a particular level (i.e., the reservoir dynamics described in Section 2.2 were disabled for the sake of this experiment). To illustrate the impact of the relative strength of all three urges, we chose only settings with

$$u_{\text{wealth}} + u_{\text{curiosity}} + u_{\text{progress}} = 1. \quad (6)$$

All 55 possible setups of urge value triples chosen from 0 to 1 in steps of 0.1 were simulated 150 times each. The resulting success rate is shown as heat maps on the

triangle spanned by all possible urge values (Fig. 10). Behavioral success depends on the motivational settings. In this scenario, all of the motivations are mandatory to complete the level: Without curiosity drive, Mario will not interact with a mushroom, will not learn how to collect coins, and will not know how to destroy the blocks at the end of the level. Without the wealth drive, Mario usually does not collect enough coins to set the necessary number of progress goals to complete the level. However, the curiosity drive also causes the collection of coins (out of curiosity about the resulting event boundary and potential conditional dependencies). Without having any progress drive, Mario does not reach the end of the level, either, because it never decides to “make progress.”

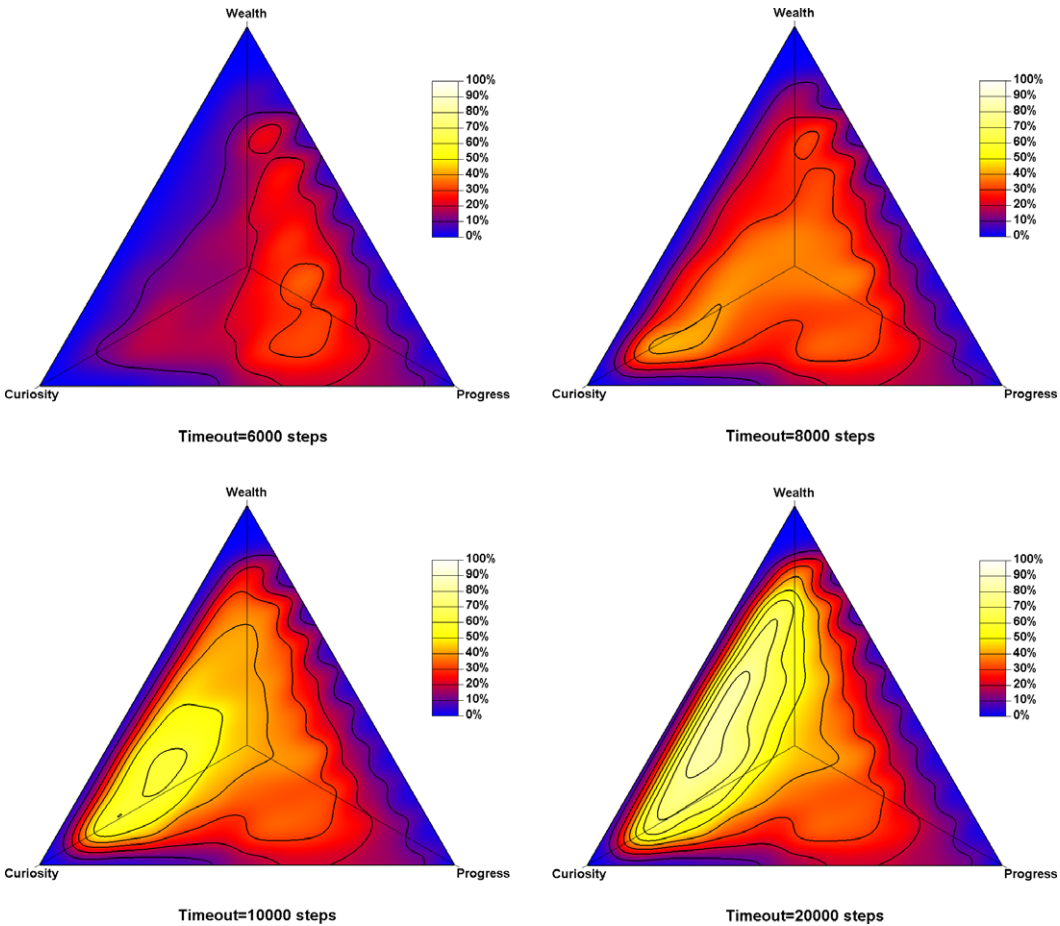


Fig. 10. Likelihood of successfully clearing the level after 6,000, 8,000, 10,000, and 20,000 simulation steps. Clearly, the more time is available, the more success is likely. Meanwhile, the motivational influences modulate the success likelihoods: With enough time available, a good balance between wealth and curiosity is optimal, while motivation for progress is relatively low, resulting in more than 90% successful runs. With less time, making progress becomes more important, while wealth and curiosity increasingly distract Mario from reaching the end in time, although they remain a requirement to obtaining knowledge and coins.

With enough time at hand, success is most likely with a good balance between wealth and curiosity, while the motivation for progress can be set relatively low. The wealth drive increases success likelihood because starvation is avoided. The curiosity drive causes interactions with relevant objects, such that necessary knowledge about collecting coins and destroying blocks is obtained. Curiosity also results in an increase in Mario's health state upon interaction with a mushroom, such that specific coins and the end of the level can be reached by destroying blocks. As time for level completion is restricted, the optimal balance of motivations at first shifts toward setups with more curiosity, since curiosity results both in gathering knowledge and—to some degree—coins. With an increasingly earlier time-out, success is increasingly restricted to setups with relatively high urge for progress, because object interactions are time-consuming and distract Mario from reaching the goal in time. Still, sufficient fractions of wealth and curiosity remain necessary to complete the level.

The evaluation thus shows that the cognitive architecture can generate self-motivated behavior, successfully solving a rather complex and slightly challenging level. Clearing the level successfully depends on reasonable balance of the individual motivational urges.

It should be emphasized that the insights gained by this evaluation are at least twofold. From the artificial intelligence side, the results show that good performance can be achieved. However, it also suggests that more strategic planning could improve performance further. From the cognitive science side, the results—as well as the previous ones and further qualitative investigations with modifications of the level structure—let us conclude that individual differences can be simulated by different weightings and dynamics of the individual motivational urges. Moreover, human-like learning and conceptual planning is possible to a certain extent. However, again one would expect that a human player improves more over time and becomes increasingly strategic. In addition, a human player builds a representation of the game into episodic and spatial memory, enabling a strategic move backward in the level.

4. Final discussion

More than 40 years ago, in the early 1970s, key figures in AI and cognitive science developed the first cognitive systems. They posed the challenge to create complete, cognitive architectures, which can interact with their world and talk about it in a human-like fashion (Newell & Simon, 1972; Winograd, 1972). While at that point chess was still considered a grand challenge, nowadays even the board game Go has been solved by an AI system, which combined deep learning, co-optimization, Monte Carlo tree search, and a few other AI techniques (Silver et al., 2016). However, none of these systems have come close to developing an understanding of their environment. The cognitive architecture SEMLINCS offers a next step toward complete embodied cognitive architectures. It is able to learn a conceptualized model of its environment from continuous sensorimotor experiences. Moreover, it interacts with the environment in a self-motivated, goal-oriented manner with the help of the developing conceptual knowledge.

4.1. Previous Super Mario model

Super Mario as a research challenge was proposed in 1990 by Gray et al. (1990), suggesting that many insights could be gained when modeling User-Nintendo interactions. John, Vera, and Newell (1994) put forward a first model of such interactions based on the GOMS principle (Goals, Operators, Methods, and Selection rules). Indeed, their system is closely related to our concrete implementation of SEMLINCS in several respects. Goals and the involved operators were encoded as hard-coded object and level interactions, including the goal of moving toward the right and to interact with blocks. Methods encoded the particular interaction types and conditional dependencies. Selection rules were mainly hard-coded and partially further optimized given expert player data. Finally, besides this “functional level” modeling, a “key-stroke level” was modeled, which simulated the actual keystrokes. Even an optimization method to improve the key-stroke-level performance was published later (Bauer & John, 1995).

Following the example of John et al. (1994), our SEMLINCS implementation also includes two knowledge and control levels. However, our lower level behavior generation mechanism is based on standard AI techniques, which can be derived in principle from formalizations of active inference. Moreover, our architecture learns higher level knowledge from the gathered sensorimotor experiences by applying the refference principle, detecting unexpected but significant game event boundaries, which are then stored in event schema-like rules. Thus, methods and operators are not hard-coded in SEMLINCS but are learned based on self-generated sensorimotor experiences, that is, based on observing autonomously generated, key-stroke-induced environmental interactions. Moreover, goals are not triggered by selection rules but by a dynamic, reservoir-based motivational system, and goal-oriented conceptual planning mechanisms.

4.2. Key modeling insights

We have not evaluated the implementation of SEMLINCS by modeling human behavior in a rigorous, quantitative fashion. Human behavior clearly varies depending on the knowledge about the game, the experience of playing it, and the actual inherent motivation—such as clearing the level as fast as possible or exploring every possible interaction with each object. These general behavioral tendencies can indeed be modeled with the SEMLINCS implementation as shown. Moreover, selective learning based on experience was exhibited. As a result, the emergence of individualized motivation- and experience-dependent agent behavior was observable. Finally, by means of linguistic dialogs, it was shown to a certain extent that the system could communicate about its world in a meaningful manner.

To realize these abilities, it was mandatory to learn higher level, somewhat conceptual, event schema rules from sensorimotor experiences. An event boundary detection mechanism was employed, which signals unexpected but lasting changes in the environment—a principle that is closely linked to mathematical formalizations of surprise and sudden, unexpectedly large free energy states (Butz, 2016; Butz et al., 2004; Gumbsch et al.,

2016). The learned event schema rules essentially constitute abstract predictive knowledge about the environment, which conceptualizes the experienced world semantics.

These rules, however, are not only useful for providing a model of the environment *per se*. Much more important, they are useful both for invoking higher level, abstract, goal-oriented planning processes and for the realization of a language interface, which enables the agent to verbalize its conceptual world knowledge. Note that with respect to planning, the architecture is closely related to principles of hierarchical reinforcement learning (Sutton, Precup, & Singh, 1999), which in turn is believed to be closely related to human action control (Botvinick & Weinstein, 2014; Botvinick, Niv, & Barto, 2009). Hierarchical reinforcement learning also addressed the challenge of generating suitable hierarchical behavioral representations (Konidaris & Barto, 2009; Vigorito & Barto, 2010), but still has no generally accepted methodology to offer. Moreover, with respect to the language interface, the rules are good for the generation of meaningful user interactions, highlighting their natural compatibility with the grammar of a natural language.

A second important insight is the methodology that was employed to create an autonomous, self-motivated behavior system. Based on the principle of autopoiesis and intrinsically motivated systems (Maturana & Varela, 1980), internal reservoirs generate behavioral urges, which translate into the generation of behavioral goals. Similar systems have been applied in the adaptive behavior community (e.g., Butz et al., 2010). Although several design choices had to be made in the actual implementation, and the evaluations focused on fixed urges, the general principle can be derived from mathematical formalizations of active inference and the resulting, anticipatory-free energy minimization mechanisms (Butz, 2016; Friston et al., 2013).

Clearly, there are still many system limitations in the current SEMLINCS implementation. Although the event-based learning was formulated in a general manner, the implementation in the game pre-coded the four types of object interactions and the given conditions. Moreover, the rule generalization mechanisms were hard-coded and could be further optimized in the near future. For example, schema learning-oriented machine learning techniques could be used (cf e.g., Butz, 2002), which are compatible with the SEMLINCS architecture.

Additionally, the spatial cognitive abilities of the system are currently mainly hard-coded and could be significantly improved. In the same manner, spatial and also episodic memory capabilities are currently fully lacking, such that the system, for example, is incapable of deciding to move back through the level, because, for example, a now important object can be found there.

Similarly, SEMLINCS currently lacks strategic planning abilities. The motivational system is still rather simplistic, choosing the “executive reservoir” anew each time. A strategy-driven, sequential choice of reservoirs would be desirable, possibly leading to the formation of more complex goals, which require the execution of a suitable sequence of goals. We believe that such mechanisms can be derived from advanced formalization of active inference, which would need to incorporate interaction effects in its predictive encodings, such as the loss of energy, changes in object distances, and changes in the internal states of the agent given certain interaction events. Moreover, such an advanced

system would need to incorporate its episodic memory into its goal-selection mechanism, as detailed above, in order to be able to develop deep behavioral strategies.

On the language side, grammar grounding may be investigated further. One very hard challenge is to learn the implemented tag-mediated syntax from scratch—possibly modeling the emergence of a sensorimotor-grounded, universal grammar (Harnad, 1990; Knott, 2012; Pastra & Aloimonos, 2012; Steels, 2008; Sugita, Tani, & Butz, 2011).

Finally, social interactions could be studied and improved further—enabling an agent, for example, to cooperate with another agent to achieve a particular goal. Particularly intriguing would be to have the agents share the gained reward or return the favor, for example, by incorporating a motivation for social interaction and a more elaborate value system. However, it is admittedly still unclear which additional system enhancements would be necessary to develop this seemingly uniquely human ability (Tomasello, 2014) with the SEMLINCS architecture.

4.3. A complete embodied cognitive architecture

More than 40 years ago, Allen Newell has emphasized the importance of creating whole cognitive architectures, with the aim of integrating many individual insights in cognitive science and psychology and thus generating deeper insights into the actual underlying cognitive processes. Clearly, the implementation of our SEMLINCS architecture is just another small step toward the creation of a complete embodied cognitive architecture, which learns conceptual structures from sensorimotor experiences, and which acts goal-directed in a strategic, intelligent, possibly human-like manner. Forty years of deep psychological and cognitive science studies have generated far more insights into individual, problem-particular cognitive processes and mechanisms than a first implementation of a new cognitive architecture ever can. Nonetheless, SEMLINCS attempts to integrate many of these insights into one whole, integrative system. Future research will show if the SEMLINCS approach is indeed enhanceable in the suggested manners.

Moreover, SEMLINCS should certainly not only be applied in the Super Mario virtual world. We are convinced that SEMLINCS is applicable in other virtual worlds for the creation of intelligent, artificial cognitive agents as well as for the modeling of human-like embodied conceptual learning, behavior, and cognition in further detail.

In conclusion, we believe that the study and development of cognitive game agents is just at its beginning. The possible insights for cognitive science facing the challenge of generating human-like cognition in game agents, which may generate a common sense-like understanding of their environment, may be very revealing. Virtual, game-like environments may be the perfect test base because (a) the simulated world is closely related to the real world, (b) it can be used to generate large amounts of data, and (c) it can be externally controlled to generate suitable experiments and system evaluations. Thus, virtual environments are very well suited to further develop complete, sensorimotor-grounded, embodied cognitive architectures.

Acknowledgments

The implementation of SEMLINCS (see Fig. 1) was developed in several student software engineering courses over the recent years at the Chair of Cognitive Modeling, University of Tübingen. Many thanks go to the enthusiastic students who were involved in the diverse subprojects and our three student assistants, Mihael Simonič, Jonas Einig, and Marcel Binz.

Notes

1. Remember that, as stated before, in this version of the Super Mario game, specific blocks are destructible only if the player's health condition is "large."
2. <http://www.youtube.com/watch?v=AplG6KnOr2Q>
3. <http://www.youtube.com/watch?v=ltPj3RIN4Nw>
4. http://www.youtube.com/watch?v=GzDt1t_iMU8

References

- Anderson, J. R. (1993). *Rules of the mind*. Hillsdale, NJ: Lawrence Erlbaum Associates.
- Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An integrated theory of the mind. *Psychological Review*, 111, 1036–1060.
- Bar, M. (2009). Predictions: A universal principle in the operation of the human brain. *Philosophical Transactions of the Royal Society B Biological Sciences*, 364 (1521), 1181–1182.
- Barsalou, L. W. (2008). Grounded cognition. *Annual Review of Psychology*, 59, 617–645.
- Bastos, A., Usrey, W., Adams, R., Mangun, G., Fries, P., & Friston, K. (2012). Canonical microcircuits for predictive coding. *Neuron*, 76(4), 695–711.
- Bauer, M. I., & John, B. E. (1995). Modeling time-constrained learning in a highly interactive task. In I.R. Katz, R. Mack, L. Marks, M.B. Rosson, & J. Nielsen. (Eds.), *Proceedings of the SIGCHI conference on human factors in computing systems* (pp. 19–26). New York, NY: ACM Press/Addison-Wesley.
- Botvinick, M., & Weinstein, A. (2014). Model-based hierarchical reinforcement learning and human action control. *Philosophical Transactions of the Royal Society of London B: Biological Sciences*, 369(1655), doi: 10.1098/rstb.2013.0480.
- Botvinick, M., Niv, Y., & Barto, A. C. (2009). Hierarchically organized behavior and its neural foundations: A reinforcement learning perspective. *Cognition*, 113(3), 262–280.
- Butz, M. V. (2002). *Anticipatory learning classifier systems*. Boston: Kluwer Academic Publishers.
- Butz, M. V. (2008). How and why the brain lays the foundations for a conscious self. *Constructivist Foundations*, 4(1), 1–42.
- Butz, M. V. (2016). Towards a unified sub-symbolic computational theory of cognition. *Frontiers in Psychology*, 7(925), doi: 10.3389/fpsyg.2016.00925.
- Butz, M. V., & Hoffmann, J. (2002). Anticipations control behavior: Animal behavior in an anticipatory learning classifier system. *Adaptive Behavior*, 10, 75–96.
- Butz, M. V., Sigaud, O., & Gérard, P. (2003). Internal models and anticipations in adaptive learning systems. In M. V. Butz, O. Sigaud, & P. Gérard (Eds.), *Anticipatory behavior in adaptive learning systems: Foundations, theories, and systems* (pp. 86–109). Berlin, Heidelberg: Springer-Verlag.

- Butz, M. V., Swarup, S., & Goldberg, D. E. (2004). Effective online detection of taskindependent landmarks. In R. S. Sutton & S. Singh (Eds.), *Online proceedings for the ICML'04 workshop on predictive representations of world knowledge* (pp. 10). Available at <https://homepage.mac.com/rssutton/ICMLWorkshop.html>. Accessed 20 12, 2011.
- Butz, M. V., Herbort, O., & Hoffmann, J. (2007). Exploiting redundancy for exible behavior: Unsupervised learning in a modular sensorimotor control architecture. *Psychological Review*, 114, 1015–1046.
- Butz, M. V., Shirinov, E., & Reif, K. L. (2010). Self-organizing sensorimotor maps plus internal motivations yield animal-like behavior. *Adaptive Behavior*, 18(3–4), 315–337.
- Clark, A. (2013). Whatever next? Predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Science*, 36, 181–253.
- Dominey, P. F. (2013). Recurrent temporal networks and language acquisition: From corticostriatal neurophysiology to reservoir computing. *Frontiers in Psychology*, 4, 500.
- Ehrenfeld, S., Herbort, O., & Butz, M. V. (2013). Modular neuron-based body estimation: Maintaining consistency over different limbs, modalities, and frames of reference. *Frontiers in Computational Neuroscience*, 7(148), doi: 10.3389/fncom.2013.00148.
- Friston, K. (2003). Learning and inference in the brain. *Neural Netw*, 16(9), 1325–1352.
- Friston, K. (2009). The free-energy principle: A rough guide to the brain? *Trends in Cognitive Sciences*, 13 (7), 293–301.
- Friston, K. (2010). The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11, 127–138.
- Friston, K., Mattout, J., & Kilner, J. (2011). Action understanding and active inference. *Biological Cybernetics*, 104(1–2), 137–160.
- Friston, K., Schwartenbeck, P., Fitzgerald, T., Moutoussis, M., Behrens, T., & Dolan, R. J. (2013). The anatomy of choice: Active inference and agency. *Frontiers in Human Neuroscience*, 7(598), doi: 10.3389/fnhum.2013.00598.
- Friston, K., Rigoli, F., Ognibene, D., Mathys, C., FitzGerald, T., & Pezzulo, G. (2015). Active inference and epistemic value. *Cognitive Neuroscience*, 6, 187–214.
- Gray, W. D., Atwood, M. E., Olson, J. S., Gray, W. D., John, B. E., Elkerton, J., & Atwood, M. E. (1990). Panel session: Real-time GOMS: Comparative modeling of a usernintendo interaction. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, 34(4), 385–386.
- Gumbsch, C., Kneissler, J., & Butz, M. V. (2016). Learning behavior-grounded event segmentations. In A. Papafragou, D. Grodner, D. Mirman & J.C. Trueswell, (Ed.), *Proceedings of the 38th Annual Meeting of the Cognitive Science Society* (pp. 1787–1792). Austin, TX: Cognitive Science Society.
- Harnad, S. (1990). The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42, 335–346.
- Haruno, M., Wolpert, D. M., & Kawato, M. (2001). Mosaic model for sensorimotor learning and control. *Neural Computation*, 13, 2201–2220.
- Herbart, J. F. (1825). *Psychologie als Wissenschaft neu gegründet auf Erfahrung, Metaphysik und Mathematik. Zweiter, analytischer Teil [Psychology as a science newly grounded on experience, metaphysics, and mathematics. second part: Analytics]*. Königsberg, Germany: August Wilhelm Unzer.
- Hoffmann, J. (1993). *Vorhersage und Erkenntnis: Die Funktion von Antizipationen in der menschlichen Verhaltenssteuerung und Wahrnehmung. [Anticipation and cognition: The function of anticipations in human behavioral control and perception]*. Göttingen, Germany: Hogrefe.
- Holst, E. & von Mittelstaedt, H. (1950). Das Refferenzprinzip (Wechselwirkungen zwischen Zentralnervensystem und Peripherie.). *Naturwissenschaften*, 37, 464–476.
- Hommel, B., Müsseler, J., Aschersleben, G., & Prinz, W. (2001). The theory of event coding (TEC): A framework for perception and action planning. *Behavioral and Brain Sciences*, 24, 849–878.
- Hwang, Y. K., & Ahuja, N. (1992). Gross motion planning—A survey.find more like this. *ACM Computing Surveys*, 24, 219–229.
- Jackendoff, R. (2002). *Foundations of language: Brain, meaning, grammar, evolution*. Oxford, UK: Oxford University Press.

- James, W. (1890). *The principles of psychology*. New York: Dover Publications.
- John, B. E., Vera, A. H., & Newell, A. (1994). Towards real-time GOMS: A model of expert behaviour in a highly interactive task. *Behaviour & Information Technology*, 13(4), 255–267.
- Jordan, M. I., & Rumelhart, D. E. (1992). Forward models: Supervised learning with a distal teacher. *Cognitive Science*, 16, 307–354.
- Kneissler, J., Drugowitsch, J., Friston, K., & Butz, M. V. (2015). Simultaneous learning and filtering without delusions: A bayes-optimal combination of predictive inference and adaptive filtering. *Frontiers in Computational Neuroscience*, 9(47), doi: 10.3389/fncom.2015.00047.
- Knott, A. (2012). *Sensorimotor cognition and natural language syntax*. Cambridge, MA: MIT Press.
- Konidaris, G., & Barto, A. (2009). Efficient skill learning using abstraction selection. *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, 21, 1107–1112.
- Lamere, P., Kwok, P., Walker, W., Gouvêa, E. B., Singh, R., Raj, B., & Wolf, P. (2003). Design of the cmu sphinx-4 decoder. *8th European Conference on Speech Communication and Technology, EUROSPEECH 2003 - INTERSPEECH 2003*, 8, 1181–1184.
- Langley, P., & Choi, D. (2006). A unified cognitive architecture for physical agents. *Proceedings of the National Conference on Artificial Intelligence*, 21, 1469–1474.
- Langley, P., Laird, J. E., & Rogers, S. (2009). Cognitive architectures: Research issues and challenges. *Cognitive Systems Research*, 10(2), 141–160.
- Levesque, H. J. (2014). On our best behaviour. *Artificial Intelligence*, 212, 27–35.
- Maturana, H., & Varela, F. (1980). *Autopoiesis and cognition: The realization of the living*. Boston, MA: Reidel.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Newell, A. (1973). You can't play 20 questions with nature and win: Projective comments on the papers of this symposium. In W. G. Chase (Ed.), *Visual information processing* (pp. 283–308). New York: Academic Press.
- Newell, A. (1990). *Unified theories of cognition: The William James lectures*. Cambridge, MA: Harvard University Press.
- Newell, A., & Simon, H. A. (1972). *Human problem solving*. Englewood Cliffs, NJ: Prentice-Hall.
- Pastr, K., & Aloimonos, Y. (2012). The minimalist grammar of action. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 367, 103–117.
- Polson, P. G., & Lewis, C. H. (1990). Theory-based design for easily learned interfaces. *Human-Computer Interaction*, 5(2–3), 191–220.
- Prinz, W. (1990). A common coding approach to perception and action. In O. Neumann & W. Prinz (Eds.), *Relationships between perception and action* (pp. 167–201). Berlin, Heidelberg: Springer-Verlag.
- Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.
- Rolf, M., Steil, J. J., & Gienger, M. (2011). Online goal babbling for rapid bootstrapping of inverse models in high dimensions. *Development and Learning (ICDL), 2011 IEEE International Conference on*, 1–8, doi: 10.1109/DEVLRN.2011.6037368.
- Rosenbloom, P. S., Laird, J. E., & Newell, A. (1992). *The soar papers: Research on integrated intelligence* (Vol. 1 and 2). Cambridge, MA: MIT Press.
- Schrodt, F., & Butz, M. V. (2016). Just imagine! Learning to emulate and infer actions with a stochastic generative architecture. *Frontiers in Robotics and AI*, 3(5), doi: 10.3389/frobt.2016.00005.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.

- Steels, L. (2008). The symbol grounding problem has been solved. So what's next? In M. de Vega, A. M. Glenberg, & A. C. Graesser (Eds.), *Symbols and embodiment: Debates on meaning and cognition* (pp. 223–244). New Haven, CT: Academic Press.
- Stock, A., & Stock, C. (2004). A short history of ideo-motor action. *Psychological Research*, 68, 176–188.
- Sugita, Y., Tani, J., & Butz, M. V. (2011). Simultaneously emerging braitenberg codes and compositionality. *Adaptive Behavior*, 19, 295–316.
- Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112, 181–211.
- Tomasello, M. (2014). *A natural history of human thinking*. Cambridge, MA: Harvard University Press.
- Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59, 433–460.
- Vigorito, C. M., & Barto, A. G. (2010). Intrinsically motivated hierarchical skill learning in structured environments. *Autonomous Mental Development, IEEE Transactions on*, 2(2), 132–143.
- Winograd, T. (1972). Understanding natural language. *Cognitive Psychology*, 3(1), 1–191.
- Wolpert, D. M., & Flanagan, J. R. (2003). Sensorimotor learning. In M. Arbib (Ed.), *The handbook of brain theory and neural networks* (2nd ed., pp. 1020–1023). Cambridge, MA: MIT Press.
- Wolpert, D. M., & Kawato, M. (1998). Multiple paired forward and inverse models for motor control. *Neural Networks*, 11, 1317–1329.
- Wörgötter, F., Agostini, A., Krüger, N., Shylo, N., & Porr, B. (2009). Cognitive agents-a procedural perspective relying on the predictability of object-actioncomplexes (OACs). *Robotics and Autonomous Systems*, 57(4), 420–432.
- Zacks, J. M., & Tversky, B. (2001). Event structure in perception and conception. *Psychological Bulletin*, 127(1), 3–21.
- Zacks, J. M., Speer, N. K., Swallow, K. M., Braver, T. S., & Reynolds, J. R. (2007). Event perception: A mind-brain perspective. *Psychological Bulletin*, 133(2), 273–293.
- Zacks, J. M., Kumar, S., Abrams, R. A., & Mehta, R. (2009). Using movement and intentions to understand human activity. *Cognition*, 112(2), 201–216.

Appendix

A. Implementation details of motivational system

Curiosity reflects the agent's incentive to learn about events and event-encoding production rules (i.e., event schemata). The drive is lowered whenever the agent interacts with an object and gathers new or validates existing event schema knowledge about the object ($\sum |\Delta P^i| \geq$ or < 0.1 , respectively). Its reservoir dynamics is defined by the equation

$$f_1(s_1^{t-1}) = s_1^{t-1} + \text{Exp}(\lambda_1), \quad (7)$$

where $\text{Exp}(\lambda_1)$ is an exponentially distributed random variable with parameter λ_1 .

Progress is Mario's predisposition to head to the right side of the level in order to complete it. The state is changed according to the horizontal distance covered (Δx) when a goal was reached. In common with curiosity, it is increased randomly over time, such that a given time constraint for the level is preserved.

The *wealth* reservoir represents the drive to gather coins. The state of the reservoir depends solely on the number of coins an agent possesses, and it does not implement internal dynamics:

$$f_3(s_3^{t-1}) = s_3^{t-1}. \quad (8)$$

The wealth state is increased by 5 in response to gathering a new coin, and decreased by 1 as cost for setting new goals, such that the agent will strive toward gathering new coins regularly.

Finally, *health* reflects the need for health and survival, where the “health” of Mario is simply reflected by a discrete value, which can be increased by consuming mushrooms or fire flowers and which decreases when bumping into an enemy from the sides. Thus, as for coins, the reservoir state depends solely on the current health value. When striving for more health, the agent will essentially attempt to find a fire flower or a mushroom (given it has had the experience that interactions with these two objects indeed increase the health value).